



Chapter 1

Making Recommendations

IN THIS CHAPTER

» Defining why recommenders are important

» Obtaining rating data

» Working with behaviors

» Using SVD to your advantage

.....

One of the oldest and most common sales techniques is to recommend something to a customer based on what you know about the customer's needs and wants. If people buy one product, they might buy another associated product if given a good reason to do so. They may not even have thought about the need for the second product until the salesperson recommends it, yet they really do need it to use the primary product. For this reason alone, most people actually like to get recommendations. Given that web pages now serve as a salesperson in many cases, recommender systems are a necessary part of any serious sales effort on the web. This chapter helps you better understand the significance of the recommender revolution in all sorts of venues.

Recommender systems serve all sorts of other needs. For example, you might see an interesting movie title, read the synopsis, and still not know whether you're likely to find it a good movie. Watching the

trailer might prove equally fruitless. Only after you see the reviews provided by others do you feel that you have enough information to make a good decision. In this chapter, you also find methods for obtaining and using rating data.

Gathering, organizing, and ranking such information is hard, though, and information overflow is the bane of the Internet. A recommender system can perform all the required work for you in the background, making the work of getting to a decision a lot easier. You may not even realize that search engines are actually huge recommender systems. The Google search engine, for instance, can provide personalized search results based on your previous search history.

Recommender systems do more than just make recommendations. After reading images and texts, machine learning algorithms can also read a person's personality, preferences, and needs, and act accordingly. This chapter helps you understand how all these activities take place by exploring techniques such as singular value decomposition (SVD).



REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0501_Recommender.ipynb` source code file for Python and the `DSPD_R_0501_Recommender.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Realizing the Recommendation Revolution

A recommender system can suggest items or actions of interest to a user, after having learned the user's preferences over time. The technology, which is based on data and machine learning techniques (both supervised and unsupervised), has appeared on the Internet for about

two decades. Today you can find recommender systems almost everywhere, and they're likely to play an even larger role in the future under the guise of personal assistants, such as Siri (developed by Apple), Amazon Alexa, Google Home, or some other artificial-intelligence-based digital assistant. The drivers for users and companies to adopt recommender systems are different but complementary:

- **Users:** Have a strong motivation to reduce the complexity of the modern world (regardless of whether the issue is finding the right product or a place to eat) and avoid information overload.
- **Companies:** Need recommender to systems provide a practical way to communicate in a personalized way with their customers and successfully push sales.



REMEMBER Recommender systems actually started as a means to handle information overload. The Xerox Palo Alto Research Center built the first recommender in 1992. Named Tapestry (see the story at <https://medium.com/the-graph/how-recommender-systems-make-their-suggestions-da6658029b76>), it handled the increasing number of emails received by center researchers. The idea of collaborative filtering was born by learning from users and leveraging similarities in preferences. The GroupLens project (<https://grouplens.org/>) soon extended recommender systems to news selection and movie recommendations (the MovieLens project, <https://movielens.org/>, whose data you initially work with in the “[Using the MovieLens sparse matrix](#)” section of Book 4, [Chapter 4](#)).

When giant players in the e-commerce sector, such as Amazon, started adopting recommender systems, the idea went mainstream and spread widely in e-commerce. Netflix did the rest by promoting recommenders as a business tool and sponsoring a competition to improve its recommender system (see <https://www.netflixprize.com/> and <https://www.thrillist.com/entertainment/nation/the-netflix-prize> for details) that involved various teams for quite a long

time. The result is an innovative recommender technology that uses SVD and Restricted Boltzmann Machines (a kind of unsupervised neural network).

However, recommender systems aren't limited to promoting products. Since 2002, a new kind of Internet service has made its appearance: social networks such as Friendster, Myspace, Facebook, and LinkedIn. These services promote exchanges between users and share information such as posts, pictures, and videos. In addition, these services help create links between people with similar interests. Search engines, such as Google, amassed user response information to offer more personalized services and understand how to match user's desires when responding to users' queries better

(<https://moz.com/learn/seo/google-rankbrain>).

Recommender systems have become so pervasive in guiding people's daily life that experts now worry about the impact on our ability to make independent decisions and perceive the world in freedom. A recommender system can blind people to other options — other opportunities — in a condition called *filter bubble*. By limiting choices, a recommender system can also have negative impacts, such as reducing innovation. You can read about this concern in the articles at

<https://dorukkilitcioglu.com/2018/10/09/recommender-filter-serendipity.html> and

<https://www.technologyreview.com/s/522111/how-to-burst-the-filter-bubble-that-protects-us-from-opposing-views/> . One detailed study of the effect, entitled “Exploring the Filter Bubble: The Effect of Using Recommender Systems on Content Diversity,” appears

on ACM at <https://dl.acm.org/citation.cfm?id=2568012> . The history of recommender systems is one of machines striving to learn about our minds and hearts, to make our lives easier, and to promote the business of their creators.

Downloading Rating Data

Getting good rating data can be hard. Later in this chapter, you use the MovieLens dataset to see how SVD can help you in creating movie recommendations. (MovieLens is a sparse matrix dataset that you can see demonstrated in Book 4, [Chapter 4](#).) However, you have other databases at your disposal. The following sections tell you more about the MovieLens dataset and describe the data logs contained in MSWeb — both of which work quite well when experimenting with recommender systems.

Navigating through anonymous web data

One of the more interesting datasets that you can use to learn about preferences is the MSWeb dataset

(<https://archive.ics.uci.edu/ml/datasets/Anonymous+Microsoft+Web+Data>).

It consists of a week's worth of anonymously recorded data from the Microsoft website with these characteristics:

- **Number of instances:** 37,711
 - **Training:** 32,711
 - **Test:** 5,000
- **Number of attributes:** 294
- **Number of users:** 32,710
- **Number of Vroots:** 285

In this case (unlike the MovieLens dataset), the recorded information is about a behavior, not a judgment, thus values are expressed in a binary form. You can download the MSWeb dataset from

<https://github.com/amirkrifa/ms-web-dataset/raw/master/anonymous-msweb.data>, get information about its structure, and explore how its values are distributed. The following code shows how to obtain the data using Python:

```
import urllib.request
```

```
import os.path
```

```
filename = "anonymous-msweb.data"

if not os.path.exists("anonymous-msweb.data"):

    url = "https://github.com/amirkrifa/ms-web-dataset/\
raw/master/anonymous-msweb.data"

urllib.request.urlretrieve(url, filename)
```



REMEMBER The technique for obtaining the file is similar to that used for the MovieLens dataset. In fact, this is a kind of CSV file, but you won't use Pandas to work with it because it has a complex dataset structure. The sections that follow describe how to work with this dataset in Python. R actually makes the process of working with the MSWeb dataset considerably easier because you can download the MSWeb dataset from the R recommenderlab library. If you want to see Python techniques in addition to those in this chapter for working with the MSWeb dataset, check out the site at <https://github.com/amirkrifa/ms-web-dataset>.

Parsing the data file

The data file contains complex data to track user behavior, and you may encounter this sort of data when performing data science tasks. It looks complicated at first, but if you break the data file down carefully, you can eventually tease out the file details. If you were to open this data file (it's text, so you can look if desired), you would find that it contains three kinds of records:

- **A:** Attributes of the particular page. Each attribute is a different page, so you could use the word page (or pages for multiples) in place of attributes, but

the example uses attributes for clarity.

- **C:** Users who are looking at the pages.
- **V:** Vroots for each of the pages. A *Vroot* is a series of grouped website pages. Together they constitute an area of the website. The binary values show whether someone has visited a certain area. (You just see a flag; you don't see how many times the user has actually visited that website area.)

Each record appears on a separate line. Consequently, you build one dictionary for each of the record types to separate one from the other, as shown here:

```
import codecs
```

```
import collections
```

```
# Open the file.
```

```
file = codecs.open(filename, 'r')
```

```
# Setup for attributes.
```

```
attribute = collections.namedtuple(
```

```
    'page', ['id', 'description', 'url'])
```

```
attributes = {}
```

```
# Setup for users
```

```
current_user_id = None
```

```
current_user_ids = []

user_visits = {}

# Setup for Vroots

page_visits = {}

# Process the data one line at a time and place
# each record in the appropriate storage unit.

for line in file:

    chunks = line.split(',')

    entry_type = chunks[0]

    if entry_type == 'A':

        type, id, ignored, description, url = chunks

        attributes[int(id)] = attribute(

            id=int(id), description=description, url=url)

    if entry_type == 'C':

        if not current_user_id == None:
```



```
user_visits[current_user_id] = set(  
    current_user_ids)  
    current_user_ids = []  
    current_user_id = int(chunks[2])  
  
    if entry_type == 'V':  
        page_id = int(chunks[1])  
        current_user_ids.append(page_id)  
        page_visits.setdefault(page_id, [])  
        page_visits[page_id].append(current_user_id)  
  
# Display the totals  
print('Total Number of Attributes: ',  
    len(attributes.keys()))  
print('Total Number of Users: ', len(user_visits.keys()))  
print('Total Number of VRoots: ', len(page_visits.keys()))
```

The code begins by setting up variables to hold information for each of the record types. It then reads the file one line at a time and determines the record type. Each record requires a different kind of process. For example, an attribute contains a page number, description, and URL. User records contain the user ID and a list of pages that

the user has visited. The Vroot entries associate pages with users. At the end of the process, you can see the number of each kind of record in the dataset.

```
Total Number of Attributes: 294
```

```
Total Number of Users: 32710
```

```
Total Number of VRoots: 285
```

The idea is that a user's visit to a certain area indicates a specific interest. For instance, when a user visits pages to learn about productivity software along with visits to a page containing terms and prices, this behavior indicates an interest in acquiring the productivity software soon. Useful recommendations can be based on such inferences about a user's desire to buy certain versions of the productivity software or bundles of different software and services.

Viewing the attributes

It's important to remember that the focus is on pages and users viewing them, so it pays to know a little something about the pages. After you parse the dataset, the following code will display the page information for you:

```
for k, v in attributes.items():  
  
    print("{:4} {:30.30} {:12}".format(  
  
        v.id, v.description, v.url))
```

When you run this code, you see all 294 attributes (pages). Here is a partial listing:

```
1287 "International AutoRoute"      "/autoroute"
```

```
1288 "library" "/library"
```

```
1289 "Master Chef Product Infor..." "/masterchef"
```

```
1297 "Central America" "/centroam"
```

```
1215 "For Developers Only Info" "/developer"
```

```
1279 "Multimedia Golf" "/msgolf"
```

```
1239 "Microsoft Consulting" "/msconsult"
```

Obtaining statistics

In addition to viewing the data, you can also perform analysis on it by various means, such as statistics. Here are some statistics you can try with the users:

```
nbr_visits = list(map(len, user_visits.values()))
```

```
average_visits = sum(nbr_visits) / len(nbr_visits)
```

```
one_visit = sum(x == 1 for x in nbr_visits)
```

```
print("Number of user visits: ", sum(nbr_visits))
```

```
print("Average number of visits: ", average_visits)
```

```
print("Users with just one visit: ", one_visit)
```

When you run this code, you see some interesting information about the users who visited the various pages:

```
Number of user visits: 98653
```

Average number of visits: 3.0159889941913787

Users with just one visit: 9994

Encountering the limits of rating data

For recommender systems to work well, they need to know about you as well as other people, both like you and different from you.

Acquiring rating data allows a recommender system to learn from the experiences of multiple customers. Rating data could derive from a judgment (such as rating a product using stars or numbers) or a fact (a binary 1/0 that simply states that you bought the product, saw a movie, or stopped browsing at a certain web page).



REMEMBER No matter the data source or type, rating data is always about behaviors. To rate a movie, you have to decide to see it, watch it, and then rate it based on your experience of seeing the movie. Actual recommender systems learn from rating data in different ways:

- **Collaborative filtering:** Matches raters based on movie or product similarities used in the past. You can get recommendations based on items liked by people similar to you or on items similar to those you like.
- **Content-based filtering:** Goes beyond the fact that you watched a movie. It examines the features relative to you and the movie to determine whether a match exists based on the larger categories that the features represent. For instance, if you are a female who likes action movies, the recommender will look for suggestions that include the intersection of these two categories.
- **Knowledge based recommendations:** Based on metadata, such as preferences expressed by users and product descriptions. It relies on machine learning and is effective when you do not have enough behavioral data to determine user or product characteristics. This is called a *cold start* and represents one of the most difficult recommender tasks because you don't have access to either collaborative filtering or content-based filtering.

The example that appears in the sections that follow performs collaborative filtering. It locates the movies that are the most similar to *Young Frankenstein*.

Considering collaborative filtering

When using collaborative filtering, you need to calculate similarity. See Chapter 14 of *Machine Learning For Dummies*, by John Paul Mueller and Luca Massaron (Wiley), for a discussion of the use of similarity measures. Another good place to look is at

<http://dataaspirant.com/2015/04/11/five-most-popular-similarity-measures-implementation-in-python/>. Apart from

Euclidean, Manhattan, and Chebyshev distances, the remainder of this section discusses cosine similarity. *Cosine similarity* measures the angular cosine distance between two vectors, which may seem like a difficult concept to grasp but is just a way to measure angles in data spaces.

The idea behind the cosine distance is to use the angle created by the two points connected to the space origin (the point where all dimensions are zero) instead. If the points are near, the angle is narrow, no matter how many dimensions are there. If they are far away, the angle is quite large. Cosine similarity implements the cosine distance as a percentage and is quite effective in telling whether a user is similar to another or whether a film can be associated to another because the same users favor it.

Obtaining the data

The code in this section assumes that you have access to the MovieLens database using the code from the “[Using the MovieLens sparse matrix](#)” section of Book 4, [Chapter 4](#). Assuming that you’re working with a new notebook, however, you need to read the data into the notebook and merge the two datasets used for this example, as shown here:

```
import pandas as pd
```

```
ratings = pd.read_csv("ml-20m/ratings.csv")
```

```
movies = pd.read_csv("ml-20m/movies.csv")
```

```
movie_data = pd.merge(ratings, movies, on="movieId")
```

```
print(movie_data.head())
```

After you perform the merge, you see a new dataset, `movie_data`, which contains the combination of ratings and movies, as shown here:

	userId	movieId	rating	timestamp	title \
0	1	2	3.5	1112486027	Jumanji (1995)
1	5	2	3.0	851527569	Jumanji (1995)
2	13	2	3.0	849082742	Jumanji (1995)
3	29	2	3.0	835562174	Jumanji (1995)
4	34	2	3.0	846509384	Jumanji (1995)
	genres				
0	Adventure Children Fantasy				
1	Adventure Children Fantasy				

```
2 Adventure|Children|Fantasy
```

```
3 Adventure|Children|Fantasy
```

```
4 Adventure|Children|Fantasy
```

All these entries are for *Jumanji* because `head()` shows only the first five entries in the `movie_data` dataset, and *Jumanji* obviously has at least five ratings. You can use the new dataset to obtain a simple statistic for the movies; however, the mean of the ratings for each movie is shown here:

```
print(movie_data.groupby('title')['rating'].mean().head())
```

This code looks rather complicated, but it isn't. Calling `groupby('title')` creates a grouping of the various movies by title. You can then access the `['rating']` column of that grouping to obtain a `mean()`. The output shows the first five entries, as shown here (note that `groupby()` automatically sorts the entries for you):

```
title
```

```
"Great Performances" Cats (1998)                2.748387
```

```
#chicagoGirl: The Social Network Takes on a...  3.666667
```

```
$ (Dollars) (1971)                                2.833333
```

```
$5 a Day (2008)                                  2.871795
```

```
$9.99 (2008)                                     3.009091
```

```
Name: rating, dtype: float64
```

The `rating` column doesn't have a title, but you see it listed on the last line as the column used to create the mean, which is of type

```
float64.
```

Massaging the data

The current MovieLens dataset is huge and cumbersome. When working with an online product, such as Google Colab (see Book 1, [Chapter 3](#) for details), the dataset might very well work in its current form.

When working with a desktop system, you need to massage the data to ensure that you actually can get the desired results. In fact, massaging the data is an essential part of performing data science tasks because you may not actually have good data. This section looks at ways that you might want to massage the MovieLens dataset to ensure good results.



WARNING Desktop setups can be particularly picky when you're working with huge data. One of the issues you can encounter when working with these datasets is memory. When performing certain tasks, such as creating the pivot table for this example, you might see `ValueError: negative dimensions are not allowed` as an output. What this really means is that your system ran out of memory. You have a number of options for countering this problem, some of the most important of which appear in this section.

You can reduce the memory requirements for working with the data by removing items that you don't really want in the analysis anyway. For this analysis, you have three extra columns: `movieId`, `timestamp`, and `genres`. In addition, a person would need to think enough of a movie to give it at least three out of five stars. Consequently, you can also get rid of the lesser value reviews using the following code:

```
reduced_movie = movie_data.loc[
```

```
movie_data['rating'] >= 3.0]
```



```

reduced_movie = reduced_movie.drop(
    columns=['movieId', 'timestamp', 'genres'])

print(reduced_movie.head())

print()

print("Original Shape: {0}, New Shape: {1}".format(
    movie_data.shape, reduced_movie.shape))

```

The reduction in size doesn't actually affect the better movies. Instead, you just lose lesser movies that would have unfavorably affected the results. The size of the `reduced_movie` dataset is significantly smaller than the original `movie_data` dataset, as shown here:

	userId	rating	title
0	1	3.5	Jumanji (1995)
1	5	3.0	Jumanji (1995)
2	13	3.0	Jumanji (1995)
3	29	3.0	Jumanji (1995)
4	34	3.0	Jumanji (1995)

```
Original Shape: (20000263, 6), New Shape: (16486759, 3)
```

The number of reviews also reflects the popularity of a movie. When a movie has few reviews, it might reflect a *cult following* — a group of devotees who don't reflect the opinion of the public at large. You can remove movies with only a few reviews using the following code:

```

reduced_movie = reduced_movie[

    reduced_movie.groupby('title')['rating'].transform(

        'size') > 3000]

print(reduced_movie.groupby('title')[

    'rating'].count().sort_values().head())

print()

print("New shape: ", reduced_movie.shape)

```

The call to `transform()` selects only movies that have a certain number of reviews — more than 3,000 of them in this case. You can use `transform()` in a huge number of ways based solely on the function you provide as input, which is the built-in `size` function in this case. Here is the result of this particular bit of trimming:

title	
Eastern Promises (2007)	3001
Triplets of Belleville, The (Les triplettes de Bel...	3003
Bad Santa (2003)	3006
Mexican, The (2001)	3010
1984 (Nineteen Eighty-Four) (1984)	3010
Name: rating, dtype: int64	

```
New shape: (12083404, 3)
```



REMEMBER The way you shape your data will affect the output of any analysis you perform. You may not get the desired results the first time, so you may end up spending a lot of time trying different shaping methods. The point is to keep trying to shape the data in various ways until you obtain a good result.



TIP A final way to save memory for analysis purposes is to clean up your variables, which can consume a lot of memory. This example uses the following code for this purpose:

```
ratings = None
```

```
movies = None
```

```
movie_data = None
```

Performing collaborative filtering

Making recommendations depends on finding the right kind of information on which to make a comparison. Of course, this is where the art of data science comes into play. If making a recommendation only involved performing analysis on data in a particular manner using a specific algorithm, anyone could do it. The art is in choosing the correct data to analyze. In this section, you use a combination of the user ID and the ratings assigned by those users to a particular movie as the means to perform collaborative filtering. In other words, you're making an assumption that people who have similar tastes in movies will rate those movies at a particular level.

After you've shaped your data, you can use it to create a pivot table. The pivot table will compare user IDs with the reviews that the user has created for particular movies. Here is the code used to create the pivot table:

```
user_rating = pd.pivot_table(
```

```
    reduced_movie,
```

```
    index='userId',
```

```
    columns='title',
```

```
    values='rating')
```

```
print(user_rating.head())
```

The results might look a little odd because the pivot table will be a sparse matrix like the sample shown here:

title	Young Frankenstein	Young Guns	Zodiac	\
-------	--------------------	------------	--------	---

userId				
--------	--	--	--	--

1	4.0	NaN	NaN	
---	-----	-----	-----	--

2	NaN	NaN	NaN	
---	-----	-----	-----	--

3	5.0	NaN	NaN	
---	-----	-----	-----	--

4	NaN	NaN	NaN	
---	-----	-----	-----	--

5	NaN	NaN	NaN	
---	-----	-----	-----	--

In this case, you see that *Young Frankenstein* is the only movie that was rated by users 1 through 5. The point is that the rows contain individual user reviews and the columns are the names of movies they reviewed.

The next step in the process is to obtain a listing of reviews for the target movie, which is *Young Frankenstein*. The following code creates a list of reviewers:

```
YF_ratings = user_rating['Young Frankenstein (1974)']

print(YF_ratings.sort_values(ascending=False).head())
```

The output of this part of the code shows that *Jumanji* isn't the most popular movie around, but it'll work for the example:

```
userId
60898    5.0
52548    5.0
101177    5.0
101198    5.0
28648    5.0

Name: Young Frankenstein (1974), dtype: float64
```

Now that you have sample data to use, you can correlate it with the pivot table as a whole. The following code outputs the movies that most closely match *Jumanji* in appeal by the users who liked *Jumanji*:

```
print(user_rating.corrwith(

    YF_ratings).sort_values(
```

```
ascending=False).head())
```

The output shows that you can derive some interesting results using collaborative filtering techniques:

```
title
```

```
Young Frankenstein (1974)      1.000000
```

```
Blazing Saddles (1974)        0.421143
```

```
Monty Python and the Holy Grail (1975)  0.300413
```

```
Producers, The (1968)         0.297317
```

```
Magnificent Seven, The (1960)  0.291847
```

```
dtype: float64
```

Even though the correlation results seem a little low (with 1.000000 being the most desirable), the names of the movies selected make sense. For example, like *Young Frankenstein*, *Blazing Saddles* is a Mel Brooks movie, and *Monty Python and the Holy Grail* is a comedy.

Leveraging SVD

A property of SVD is to compress the original data at such a level and in such a smart way that, in certain situations, the technique can actually create new meaningful and useful features, not just compressed variables. The following sections help you understand what role SVD plays in recommender systems.

Considering the origins of SVD

SVD is a method from linear algebra that can decompose an initial matrix into the multiplication of three derived matrices. The three de-

rived matrices contain the same information as the initial matrix, but in a way that expresses any redundant information (expressed by statistical variance) only once. The benefit of the new variable set is that the variables have an orderly arrangement according to the initial variance portion contained in the original matrix.

SVD builds the new features using a weighted summation of the initial features. It places features with the most variance leftmost in the new matrix, whereas features with the least or no variance appear on the right side. As a result, no correlation exists between the features. (Correlation between features is an indicator of information redundancy, as explained in the previous paragraph.) Here's the formulation of SVD:

$$A = U * D * V^T$$

For compression purposes, you need to know only about matrices U and D , but examining the role of each resulting matrix helps you understand the values better, starting with the origin. A is a matrix $n \times p$, where n is the number of examples and p is the number of variables. As an example, consider a matrix containing the purchase history of n customers, who bought something in the p range of available products. The matrix values are populated with quantities that customers purchased. As another example, imagine a matrix in which rows are individuals, columns are movies, and the content of the matrix is a movie rating (which is exactly what the MovieLens dataset contains).

After the SVD computation completes, you obtain the U , S , and V matrices. U is a matrix of dimensions n by k , where k is p , exactly the same dimensions of the original matrix. It contains the information about the original rows on a reconstructed set of columns. Therefore, if the first row on the original matrix is a vector of items that Mr. Smith bought, the first row of the reconstructed U matrix will still represent Mr. Smith, but the vector will have different values. The new U

matrix values are a weighted combination of the values in the original columns.

Of course, you might wonder how the algorithm creates these combinations. The combinations are devised to concentrate the most variance possible on the first column. The algorithm then concentrates most of the residual variance in the second column, with the constraint that the second column is uncorrelated with the first one, thereby distributing the decreasing residual variance to each column in succession. By concentrating the variance in specific columns, the original features that were correlated are summed into the same columns of the new U matrix, thus cancelling any previous redundancy present. As a result, the new columns in U don't have any correlation between themselves, and SVD distributes all the original information in unique, nonredundant features. Moreover, given that correlations may indicate causality (but correlation isn't causation; it can simply hint at it — a necessary but not sufficient condition), cumulating the same variance creates a rough estimate of the variance's root cause.

V is the same as the U matrix, except that its shape is $p \times k$ and it expresses the original features with new cases as a combination of the original examples. This means that you'll find new examples composed of customers with the same buying habits. For instance, SVD compresses people buying certain products into a single case that you can interpret as a homogeneous group or as an archetypal customer.

In such reconstruction, D, a diagonal matrix (only the diagonal has values) contains information about the amount of variance computed and stored in each new feature in the U and V matrices. By cumulating the values along the matrix and making a ratio with the sum of all the diagonal values, you can see that the variance is concentrated on the first leftmost features, while the rightmost are almost zero or an insignificant value. Therefore, an original matrix with 100 features can be decomposed and have an S matrix whose first 10 newly recon-

structured features represent more than 90 percent of the original variance.

SVD has many optimizing variants with slightly different objectives. The core functions of these algorithms are similar to SVD. Principal component analysis (PCA) focuses on common variance. It's the most popular algorithm and is used in machine learning preprocessing applications.



TIP

A great SVD property is that the technique can create new meaningful and useful features, not just compressed variables, as a by-product of compression in certain situations. In this sense, you can consider SVD a feature-creation technique.

Understanding the SVD connection

If your data contains hints and clues about a hidden cause or motif, an SVD can put them together and offer you proper answers and insights. That is especially true when your data consists of interesting pieces of information like the ones in the following list:

- **Text in documents hints at ideas and meaningful categories:** Just as you can make up your mind about discussion topics by reading blogs and news-groups, so can SVD help you deduce a meaningful classification of groups of documents or the specific topics being written about in each of them.
- **Reviews of specific movies or books hint at your personal preferences and larger product categories:** If you say on a rating site that you loved the original *Star Trek* series collection, the algorithm can easily determine what you like in terms of other films, consumer products, or even personality types.

An example of a method based on SVD is latent semantic indexing (LSI), which has been successfully used to associate documents and words based on the idea that words, though different, tend to have the

same meaning when placed in similar contexts. This type of analysis suggests not only synonymous words but also higher grouping concepts. For example, an LSI analysis on some sample sports news may group baseball teams of the major league based solely on the co-occurrence of team names in similar articles, without any previous knowledge of what a baseball team or the major league are.

Other interesting applications for data reduction are systems for generating recommendations about the things you may like to buy or know more about. You likely have quite a few occasions to see recommenders in action. On most e-commerce websites, after logging in, visiting some product pages, and rating or putting a product into your electronic basket, you see other buying opportunities based on other customers' previous experiences. (As mentioned previously, this method is called *collaborative filtering*.) SVD can implement collaborative filtering in a more robust way, relying not just on information from single products but also on the wider information about a product in a set. For example, collaborative filtering can determine not only that you liked the film *Raiders of the Lost Arc* but also that you generally like all action and adventure movies.

You can implement collaborative recommendations based on simple means or frequencies calculated on other customers' sets of purchased items or on ratings using SVD. This approach helps you reliably generate recommendations even in the case of products that the vendor seldom sells or that are quite new to users.

Table of contents collapsed